

MI GFX API

Version 3.2

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	11/30/2020
	Added PROCFS introduction	08/25/2021
3.1	Added more YUV442 format	03/20/2022
3.2	Added bitblt mode	05/06/2022

TABLE OF CONTENTS

REVISION HISTORY.....	i
TABLE OF CONTENTS.....	ii
1. 概述.....	1
1.1. 模块说明	1
1.2. 调用流程框图	4
1.3. 关键字说明	5
2. API 参考.....	6
2.1. MI_GFX_Open	7
2.2. MI_GFX_Close	8
2.3. MI_GFX_WaitAllDone	9
2.4. MI_GFX_QuickFill	10
2.5. MI_GFX_GetARGB8888To1555AlphaThreshold	11
2.6. MI_GFX_SetARGB8888To1555AlphaThreshold	12
2.7. MI_GFX_BitBlit	13
2.8. MI_GFX_SetPalette	15
2.9. MI_GFX_CreateDev	16
2.10. MI_GFX_DestroyDev	17
2.11. MI_GFX_GetARGB1555To8888AlphaValue	18
2.12. MI_GFX_SetARGB1555To8888AlphaValue	18
2.13. MI_GFX_GetInitialScalingCoeff	20
2.14. MI_GFX_SetInitialScalingCoeff	21
2.15. MI_GFX_DrawLine	21
3. GFX 数据类型.....	24
3.1. MI_GFX_ColorFmt_e	25
3.2. MI_GFX_Surface_t	25
3.3. MI_GFX_Rect_t	26
3.4. MI_GFX_ColorKeyOp_e	27
3.5. MI_GFX_ColorKeyValue_t	28
3.6. MI_GFX_ColorKeyInfo_t	28
3.7. MI_GFX_Rotate_e	29
3.8. MI_GFX_Mirror_e	29
3.9. MI_GFX_DfbBldOp_e	30
3.10. MI_Gfx_DfbBlendFlags_e	31
3.11. MI_GFX_BitBltMode_e	32
3.12. MI_GFX_Opt_t	33
3.13. MI_GFX_PaletteEntry_t	34
3.14. MI_GFX_Palette_t	34
3.15. MI_GFX_DevAttr_t	35
3.16. MI_GFX_Point_t	36
3.17. MI_GFX_Line_t	36
3.18. MI_GFX_DEV	37
4. GFX 错误码.....	38
5. PROCFS 介绍.....	39

5.1. cat 39

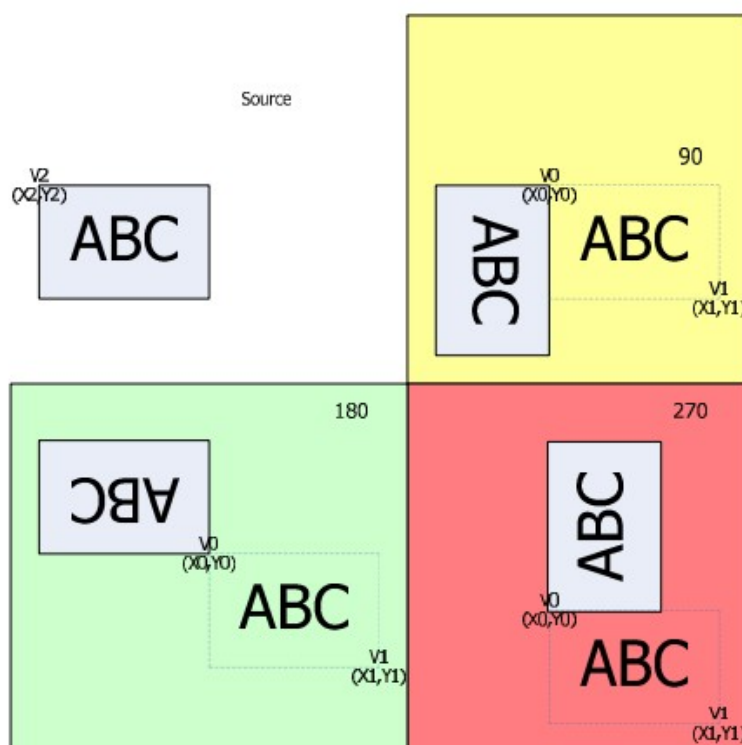
1. 概述

1.1. 模块说明

GFX（Graphic Engine）硬件为画 UI 提供快速的图形绘制功能，主要有矩形色彩填充、位图搬移（支持缩放、旋转、镜像翻转、格式转换、alpha 混合叠加、Color Key 等操作）。目前仅 Taiyaki、Takoyaki、Tiramisu、Ikayaki、Muffin、Mochi 系列芯片支持 GFX。

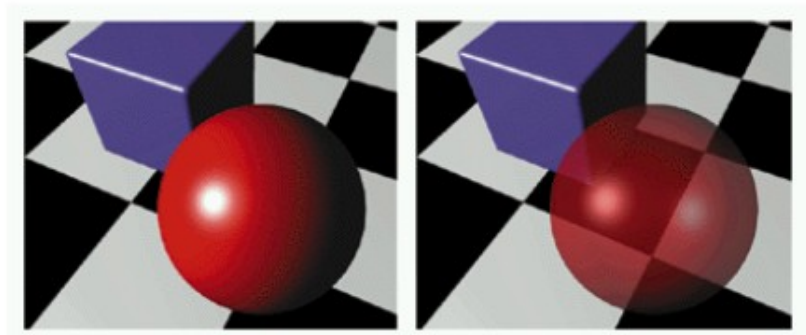
Rotate: HW 支持 90/180/270 度旋转

Rotate

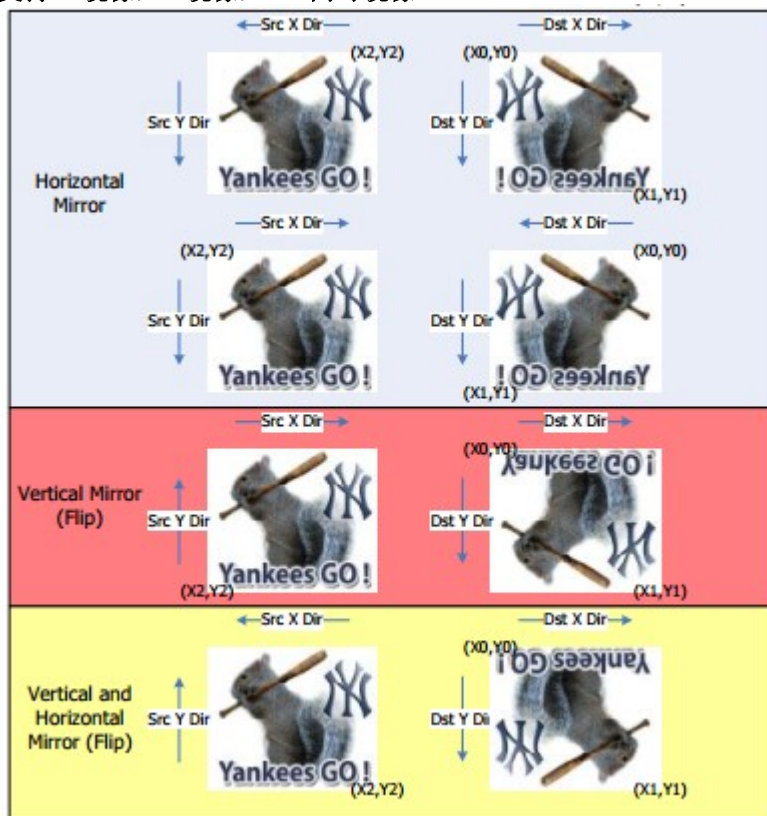


Alpha 混合叠加: 支持正常的 2D alpha 操作

Alpha Blending

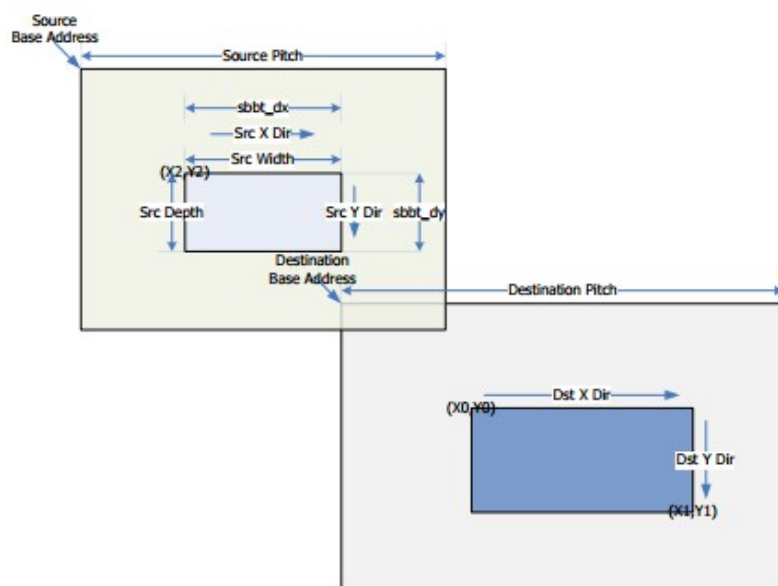


镜像：支持 **H** 镜像，**V** 镜像，**HV** 同时镜像



缩放：支持在 **bitblt** 的时候缩放

Stretch Bitblt

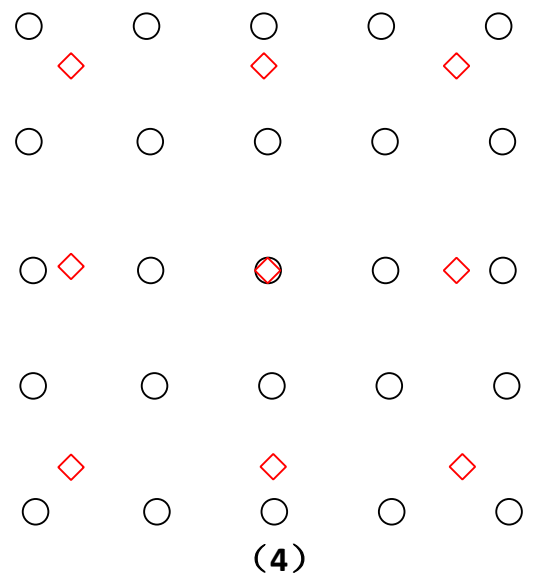
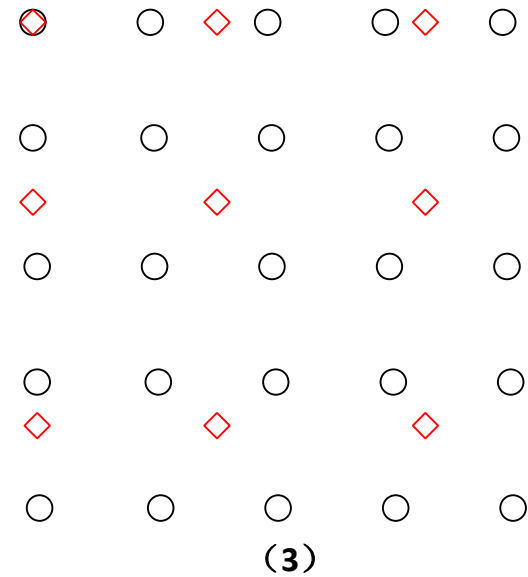
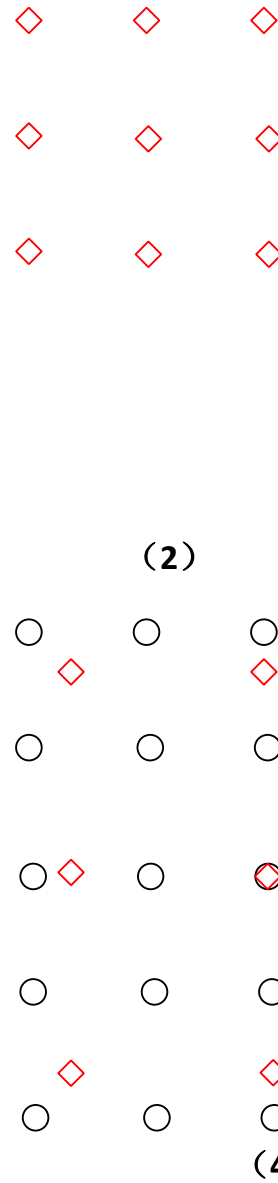
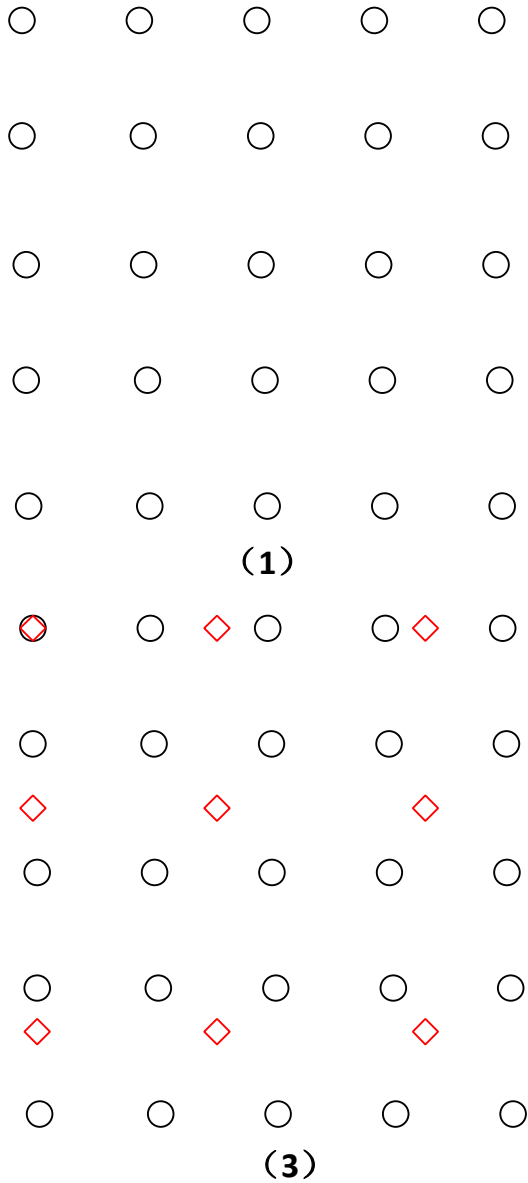


在利用双线性插值进行缩放，会涉及到图像的几何中心偏移的概念。以下图为例，当需要将 **5x5** 的图像（如图(1)）缩小到 **3x3** 的图像（如图(2)）。当不设置偏移的情况下，缩放后图像和原图像的像素对应关系如图(3)会出现整体偏左，图(4)为有设置图像几何中心偏移的对应关系。

图像几何中心:

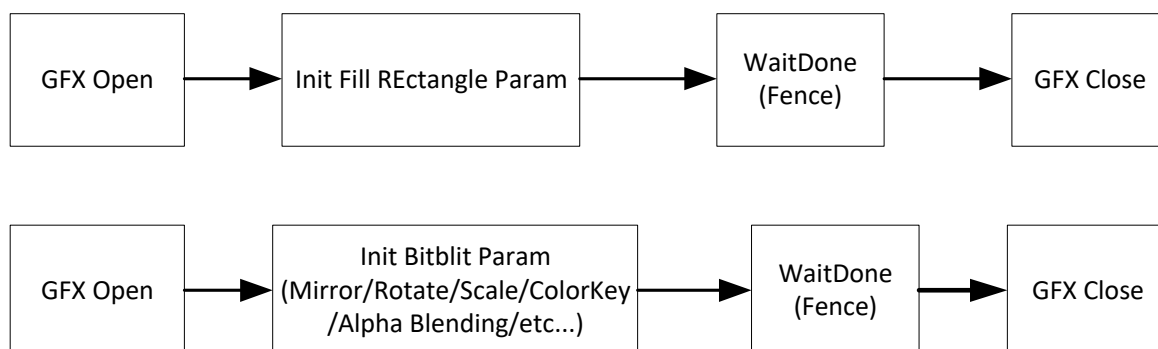
$$\text{SrcX} = (\text{dstX} + \text{coeff}) * (\text{srcWidth}/\text{dstWidth}) - \text{coeff}$$

$$\text{SrcY} = (\text{dstY} + \text{coeff}) * (\text{srcHeight}/\text{dstHeight}) - \text{coeff}$$

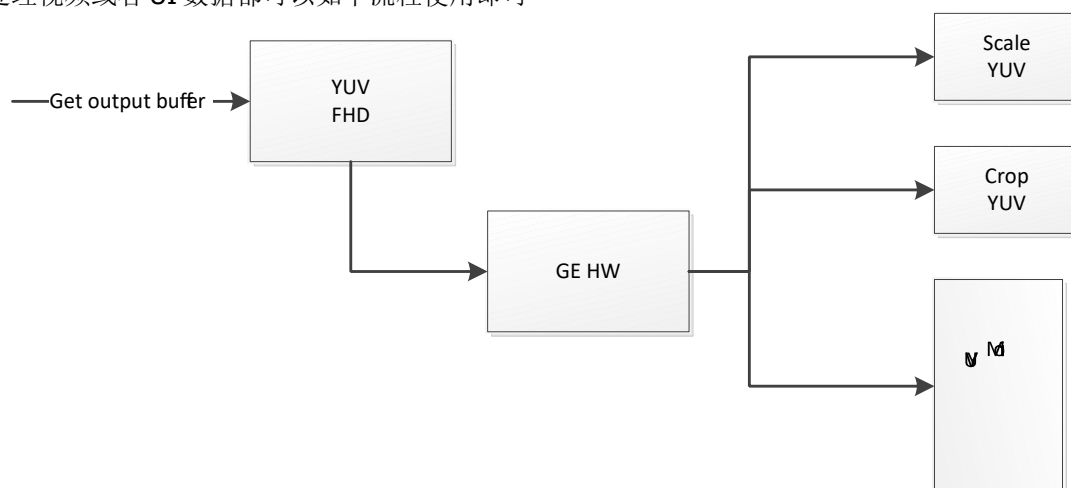


其他操作就不一一罗列，和标准的 2D 加速基本一致。

1.2. 调用流程框图



如果想用 GE 来处理视频或者 UI 数据都可以如下流程使用即可



1.3. 关键字说明

- **Fence**
对应一个 GFX 的操作需要等待的状态，会阻塞等待，但是内部也会有超时机制用来防呆。
- **Colorkey**
关键色，比如在做搬移的时候可以去掉位图中的某一个颜色，这就可以把这个颜色设置为关键色。
- **Pitch**
每行像素占的字节数， $(\text{像素位数}/8) * \text{Width}$ ，也对应常用的 Stride、LineLength。

2. API 参考

该功能模块提供以下 API:

API名	功能
MI_GFX_Open	打开 GFX 设备
MI_GFX_Close	关闭 GFX 设备
MI_GFX_WaitAllDone	等待 GFX 的某个/所有任务完成
MI_GFX_QuickFill	向任务中添加快速填充操作
MI_GFX_Bitblit	向任务中添加对光栅位图进行有附加功能的搬移操作
MI_GFX_SetPalette	设置索引颜色格式的调色板
MI_GFX_CreateDev	初始化GFX设备
MI_GFX_DestroyDev	反初始化GFX设备
MI_GFX_GetARGB8888To1555AlphaThreshold	获取ARGB8888转ARGB1555的alpha判决阈值
MI_GFX_SetARGB8888To1555AlphaThreshold	设置ARGB8888转ARGB1555的alpha判决阈值
MI_GFX_GetARGB1555To8888AlphaValue	获取ARGB1555转ARGB8888的alpha转换值
MI_GFX_SetARGB1555To8888AlphaValue	设置ARGB1555转ARGB8888的alpha转换值
MI_GFX_GetInitialScalingCoeff	获取初始缩放系数
MI_GFX_SetInitialScalingCoeff	设置初始缩放系数
MI_GFX_DrawLine	向任务中添加画线操作

2.1. MI_GFX_Open

➤ 功能
调用此接口打开GFX 设备并初始化硬件设备。

➤ 语法
MI_ S32 MI_GFX_Open([MI_GFX_DEV](#) GfxDevId);

➤ 形参

参数名称	描述	输入/输出
GfxDevId	GFX设备号	输入

➤ 返回值

返回值 {
MI_SUCCESS 打开设备成功。
初始化失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_gfx.h mi_gfx_datatype.h
库文件: libmi_gfx.a/libmi_gfx.so

※ 注意

- 1、在进行 GFX相关操作前应该首先调用此接口，保证 GFX 设备处于打开状态。
- 2、GFX设备初始化一次后，再次调用将返回已初始化，可重复调用。

➤ 举例

```
1. MI_S32 s32Ret = 0;
2. #define DEFAULT_GFX_DEV_ID (0)
3. #define DEFAULT_SOC_ID (0)
4.
5. MI_SYS_Init(DEFAULT_SOC_ID);
6. /* open GFX device*/
7. s32Ret = MI_GFX_Open(DEFAULT_GFX_DEV_ID);
8. if (MI_SUCCESS != s32Ret)
9. {
10.     return -1;
11. }
12. /* close GFX device*/
13. MI_GFX_Close(DEFAULT_GFX_DEV_ID);
14. MI_SYS_Exit(DEFAULT_SOC_ID);
```

2.2. MI_GFX_Close

➤ 功能

调用此接口关闭 GFX设备。

➤ 语法

MI_S32 MI_GFX_Close([MI_GFX_DEV](#) GfxDevId);

➤ 形参

参数名称	描述	输入/输出
GfxDevId	GFX设备号	输入

➤ 返回值

返回值 {
· MI_SUCCESS 成功销毁GFX模块。
· 销毁GFX模块失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_gfx.h mi_gfx_datatype.h
库文件: libmi_gfx.a/libmi_gfx.so

※ 注意

- 调用 [MI GFX Open](#) 与 [MI GFX Close](#) 的次数需要对应，可以重复调用，Open的次数和Close的次数一致时，GFX的api将不再能调用。

➤ 举例

无。

➤ 相关主题

无。

2.3. MI_GFX_WaitAllDone

➤ 功能

调用此接口等待 GFX 的所有任务完成或者等待指定的任务完成。

➤ 语法

MI_S32 MI_GFX_WaitAllDone([MI GFX DEV](#) GfxDevId, MI_BOOL bWaitAllDone, MI_U16 u16TargetFence);

➤ 形参

参数名称	描述	输入/输出
GfxDevId	GFX设备号	输入
bWaitAllDone	是否等待所有GFX任务完成	输入
u16TargetFence	等待指定的fence完成，在bWaitAllDone为FALSE时才有效	输入

➤ 返回值

返回值 {
· MI_SUCCESS 等待的GFX任务都已经完成。
· 等待GFX任务完成失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_gfx.h mi_gfx_datatype.h
库文件: libmi_gfx.a/libmi_gfx.so

- 注意
 - 此接口为阻塞接口，会阻塞等待所有的 GFX 任务完成。
- 举例

无。

2.4. MI_GFX_QuickFill

- 功能

向目标位图pstDst中的目标区域pstDstRect，填入值为u32ColorVal的颜色，实现颜色填充的功能。

- 语法


```
MI_S32 MI_GFX_QuickFill(MI_GFX_DEV GfxDevId, MI_GFX_Surface_t *pstDst, MI_GFX_Rect_t *pstDstRect, MI_U32 u32ColorVal, MI_U16 *pu16Fence);
```

- 形参

参数名称	描述	输入/输出
GfxDevId	GFX设备号	输入
pstDst	目标位图。	输入
pstDstRect	目标位图操作区域。	输入
u32ColorVal	填充颜色值。	输入
pu16Fence	返回的Fence指针	输出

- 返回值

{

MI_SUCCESS 成功添加QuickFill行为的任务到HW队列。

填充色块失败，详情参照[错误码](#)。

- 依赖

头文件：mi_gfx.h mi_gfx_datatype.h
库文件：libmi_gfx.a/libmi_gfx.so

- 注意
 - 在调用此接口前应保证调用 [MI_GFX_Open](#) 打开 GFX 设备。
 - 目标位图操作区域不能超出目标位图的大小。
 - 如果是目标位图的像素格式为RGB或ARGB，u32ColorVal从高位到低位，依次是A8：R8：G8：B8，如果目标位图的像素格式为YUYV422，则A8则无效，R对应Y，G对应U，B对应V。

➤ 举例

```

1. #define DEFAULT_GFX_DEV_ID (0)
2. #define DEFAULT_SOC_ID (0)
3. FILE *fp = NULL;
4. MI_PHY phyAddr;
5. void *pVirAddr = NULL;
6. MI_GFX_Surface_t stDst;
7. MI_GFX_Rect_t stDstRect;
8. MI_U32 u32ColorVal = 0xffff0000;
9. MI_U16 u16TargetFence;
10.
11. fp = fopen(SRC_FILE_NAME, "wb");
12.
13. ExecFunc(MI_SYS_Init(DEFAULT_SOC_ID), MI_SUCCESS);
14. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0", SRC_WIDTH*SRC_HEIGHT*2, &phyAddr), MI_SUCCESS);
15. ExecFunc(MI_SYS_Mmap(phyAddr, SRC_WIDTH*SRC_HEIGHT*2, &pVirAddr, FALSE), MI_SUCCESS);
16. memset(pVirAddr, 0x22, SRC_WIDTH*SRC_HEIGHT*2);
17.
18. ExecFunc(MI_GFX_Open(DEFAULT_GFX_DEV_ID), MI_SUCCESS);
19.
20. //fillrect
21. memset(&stDst, 0x0, sizeof(stDst));
22. stDst.eColorFmt = E_MI_GFX_FMT_ARGB1555;
23. stDst.u32Width = SRC_WIDTH;
24. stDst.u32Height = SRC_HEIGHT;
25. stDst.u32Stride = SRC_WIDTH * 2;
26. stDst.phyAddr = phyAddr;
27.
28. memset(&stDstRect, 0x0, sizeof(stDstRect));
29. stDstRect.s32Xpos = 100;
30. stDstRect.s32Ypos = 100;
31. stDstRect.u32Width = 100;
32. stDstRect.u32Height = 100;
33. ExecFunc(MI_GFX_QuickFill(DEFAULT_GFX_DEV_ID, &stDst, &stDstRect, u32ColorVal, &u16TargetFence), MI_SUCCESS);
34. ExecFunc(MI_GFX_WaitAllDone(DEFAULT_GFX_DEV_ID, FALSE, u16TargetFence), MI_SUCCESS);
35. if (NULL != fp)
36. {
37.     fwrite(pVirAddr, 1, SRC_WIDTH*SRC_HEIGHT*2, fp);
38.     fclose(fp);
39.     fp = NULL;
40. }
41. ExecFunc(MI_SYS_Munmap(pVirAddr, SRC_WIDTH*SRC_HEIGHT*2), MI_SUCCESS);
42. ExecFunc(MI_SYS_MMA_Free(phyAddr), MI_SUCCESS);
43.
44. ExecFunc(MI_GFX_Close(DEFAULT_GFX_DEV_ID), MI_SUCCESS);
45. ExecFunc(MI_SYS_Exit(DEFAULT_SOC_ID), MI_SUCCESS);

```

2.5. MI_GFX_GetARGB8888To1555AlphaThreshold

➤ 功能

获取 alpha 判决阈值。

当源位图和目标位图做 bitblit 操作时，不管源位图和目标位图的像素格式是什么，硬件都会各生成一个像素格式为 ARGB8888 的中间位图，再将中间位图的运算结果输出到目标位图。当目标图片像素格式为 ARGB1555 时，若中间位图运算后某像素的 alpha 值小于此阈值，则输出到目标位图的该像素 alpha 位取0；大于或等于此阈值，该像素的 alpha 位取 1。

➤ 语法

MI_S32 MI_GFX_GetARGB8888To1555AlphaThreshold([MI_GFX_DEV](#) GfxDevId, MI_U8 *pu8ThresholdValue);

➤ 形参

参数名称	描述	输入/输出
GfxDevId	GFX设备号	输入
pu8ThresholdValue	指向 alpha 判决阈值的指针	输出

➤ 返回值

返回值 {
MI_SUCCESS 成功获取到alpha阈值。
获取alpha阈值失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_gfx.h mi_gfx_datatype.h
库文件: libmi_gfx.a/libmi_gfx.so

➤ 注意

无。

➤ 举例

无。

2.6. MI_GFX_SetARGB8888To1555AlphaThreshold

➤ 功能

设置 alpha 判决阈值。

当源位图和目标位图做 bitblit 操作时，不管源位图和目标位图的像素格式是什么，硬件都会各生成一个像素格式为 ARGB8888 的中间位图，再将中间位图的运算结果输出到目标位图。当目标图片像素格式为 ARGB1555 时，若中间位图运算后某像素的 alpha 值小于此阈值，则输出到目标位图的该像素 alpha 位取0；大于或等于此阈值，该像素的 alpha 位取 1。

➤ 语法

MI_S32 MI_GFX_SetARGB8888To1555AlphaThreshold([MI_GFX_DEV](#) GfxDevId, MI_U8 u8ThresholdValue);

➤ 形参

数名称	描述	输入/输出
GfxDevId	GFX设备号	输入
u8ThresholdValue	alpha 判决阈值	输入

➤ 返回值

返回值 {
· MI_SUCCESS 成功设置alpha阈值。
· 设置alpha阈值失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_gfx.h mi_gfx_datatype.h
库文件: libmi_gfx.a/libmi_gfx.so

➤ 注意

无。

➤ 举例

无。

2.7. MI_GFX_BitBlit

➤ 功能

将源位图（pstSrc）与目标位图（pstDst）的指定区域（pstSrcRect、pstDstRect）进行运算，将运算后的位图拷贝到目标位图（pstDst）的指定区域（pstDstRect）中。

➤ 语法

MI_S32 MI_GFX_BitBlit([MI_GFX_DEV](#) GfxDevId, [MI_GFX_Surface_t](#) *pstSrc, [MI_GFX_Rect_t](#) *pstSrcRect, [MI_GFX_Surface_t](#) *pstDst, [MI_GFX_Rect_t](#) *pstDstRect, [MI_GFX_Opt_t](#) *pstOpt, MI_U16 *pu16Fence);

➤ 形参

参数名称	描述	输入/输出
GfxDevId	GFX设备号	输入
pstSrc	源位图。	输入
pstSrcRect	源位图操作区域。	输入
pstDst	目标位图。	输入
pstDstRect	目标位图操作区域。	输入
pstOpt	运算参数设置结构。	输入
pu16Fence	返回的Fence指针	输出

➤ 返回值

返回值 {
MI_SUCCESS 初始化成功。
返回失败，详情参照[错误码](#)。

➤ 依赖

头文件：mi_gfx.h mi_gfx_datatype.h
库文件：libmi_gfx.a/libmi_gfx.so

➤ 注意

- 在调用此接口前应保证调用 [MI_GFX_Open](#) 打开 GFX 设备。
- 当源位图操作区域与目标位图操作区域尺寸不一致时，源区域按照目标区域比例进行缩放后，再与目标区域做运算。
- clip 操作时若为区域内 clip，则裁减区域必须与操作区域有公共交集，否则会返回错误。
- [MI_GFX_Opt_t](#) 结构中存放有 GFX 运算功能的配置信息，如：是否作色键（colorkey）及 colorkey 的配置值；是否作区域裁减（clip 操作）及指定 clip 区域；是否镜像、是否进行 alpha 混合；插值方式等信息。上述的操作可以同时使能。

➤ 举例

```
1. #define DEFAULT_GFX_DEV_ID (0)
2. #define DEFAULT_SOC_ID (0)
3.
4. FILE *fp = NULL;
5. FILE *dstfp = NULL;
6. MI_PHY phyAddr, phyAddr2;
7. void *pVirAddr = NULL, *pVirAddr2 = NULL;
8. MI_GFX_Surface_t stSrc, stDst;
9. MI_GFX_Rect_t stSrcRect, stDstRect;
10. MI_U16 u16TargetFence;
11.
12. fp = fopen(SRC_FILE_NAME, "wb");
13. dstfp = fopen(DST_FILE_NAME, "wb");
14.
15. ExecFunc(MI_SYS_Init(DEFAULT_SOC_ID), MI_SUCCESS);
16. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0", SRC_WIDTH*SRC_HEIGHT*2, &phyAddr), MI_SUCCESS);
17. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0", DST_WIDTH*360*2, &phyAddr2), MI_SUCCESS);
18.
19. //MI_U32 phySrcAddr, phyDstAddr;
20. ExecFunc(MI_GFX_Open(DEFAULT_GFX_DEV_ID), MI_SUCCESS);
21. ExecFunc(MI_SYS_Mmap(phyAddr, SRC_WIDTH*SRC_HEIGHT*2, &pVirAddr, FALSE), MI_SUCCESS);
22. memset(pVirAddr, 0x22, SRC_WIDTH*SRC_HEIGHT*2);
23.
24. ExecFunc(MI_SYS_Mmap(phyAddr2, DST_WIDTH*DST_HEIGHT*2, &pVirAddr2, FALSE), MI_SUCCESS);
25. memset(pVirAddr2, 0x0F, DST_WIDTH*DST_HEIGHT*2);
26.
27. //bitblit
28. memset(&stSrc, 0x0, sizeof(stSrc));
29. stSrc.eColorFmt = E_MI_GFX_FMT_ARGB1555;
30. stSrc.u32Width = SRC_WIDTH;
31. stSrc.u32Height = SRC_HEIGHT;
32. stSrc.u32Stride = SRC_WIDTH * 2;
33. stSrc.phyAddr = phyAddr;
34.
35. memset(&stSrcRect, 0x0, sizeof(stSrcRect));
```

```

36. stSrcRect.s32Xpos = 100;
37. stSrcRect.s32Ypos = 100;
38. stSrcRect.u32Width = 300;
39. stSrcRect.u32Height = 300;
40.
41. memset(&stDst, 0x0, sizeof(stDst));
42. stDst.eColorFmt = E_MI_GFX_FMT_ARGB1555;
43. stDst.u32Width = DST_WIDTH;
44. stDst.u32Height = DST_HEIGHT;
45. stDst.u32Stride = DST_WIDTH * 2;
46. stDst.phyAddr = phyAddr2;
47.
48. memset(&stDstRect, 0x0, sizeof(stDstRect));
49. stDstRect.s32Xpos = 200;
50. stDstRect.s32Ypos = 100;
51. stDstRect.u32Width = 200;
52. stDstRect.u32Height = 100;
53.
54. ExecFunc(MI_GFX_BitBlit(DEFAULT_GFX_DEV_ID, &stSrc, &stSrcRect, &stDst, &stDstRect, NULL, &u16TargetFence
), MI_SUCCESS);
55. ExecFunc(MI_GFX_WaitAllDone(DEFAULT_GFX_DEV_ID, FALSE, u16TargetFence), MI_SUCCESS);
56. if (NULL != dstfp)
57. {
58.     fwrite(pVirAddr2, 1, DST_WIDTH*DST_HEIGHT*2, dstfp);
59.     fclose(dstfp);
60.     dstfp = NULL;
61. }
62.
63. ExecFunc(MI_SYS_Munmap(pVirAddr, SRC_WIDTH*SRC_HEIGHT*2), MI_SUCCESS);
64. ExecFunc(MI_SYS_MMA_Free(phyAddr), MI_SUCCESS);
65.
66. ExecFunc(MI_SYS_Munmap(pVirAddr2, DST_WIDTH*DST_HEIGHT*2), MI_SUCCESS);
67. ExecFunc(MI_SYS_MMA_Free(phyAddr2), MI_SUCCESS);
68.
69. ExecFunc(MI_GFX_Close(DEFAULT_GFX_DEV_ID), MI_SUCCESS);
70. ExecFunc(MI_SYS_Exit(DEFAULT_SOC_ID), MI_SUCCESS);

```

2.8. MI_GFX_SetPalette

➤ 功能

为索引颜色格式(I1/I2/I4/I8)设置调色板

➤ 语法

MI_S32 MI_GFX_SetPalette([MI_GFX_DEV](#) GfxDevId, [MI_GFX_ColorFmt_e](#) eColorFmt,
[MI_GFX_Palette_t](#) *pstPalette);

➤ 形参

参数名称	描述	输入/输出
GfxDevId	GFX设备号	输入
eColorFmt	调色板对应的颜色格式: MI_GFX_ColorFmt_e 。	输入
pstPalette	调色板数据。	输入

➤ 返回值

返回值 {
MI_SUCCESS 初始化成功。
返回失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_gfx.h mi_gfx_datatype.h
库文件: libmi_gfx.a/libmi_gfx.so

➤ 注意

- 在调用此接口前应保证调用 [MI GFX Open](#) 打开 GFX 设备。
- pstPalette 是一个容量为 256 的 [MI GFX PaletteEntry t](#) 数组，每一个 [MI GFX PaletteEntry t](#) 代表一种颜色，对应在数组中的下标代表了颜色的索引。但 I1/I2/I4 与 I8 各自对应一个 Palette，前者最大可支持 16 种颜色，后者最大可支持 256 种颜色。通过 eColorFmt 参数来区分设置的是哪一个 Palette。

➤ 举例

```
1. #define DEFAULT_GFX_DEV_ID (0)
2. MI_GFX_Palette_t myPal;
3. MI_GFX_PaletteEntry_t index_0;
4. MI_GFX_PaletteEntry_t index_1;
5. index_0.RGB.u8A = 0x40;
6. index_0.RGB.u8R = 0xFF;
7. index_0.RGB.u8G = 0;
8. index_0.RGB.u8B = 0;
9. index_1.RGB.u8A = 0x40;
10. index_1.RGB.u8R = 0;
11. index_1.RGB.u8G = 0xFF;
12. index_1.RGB.u8B = 0;
13.
14. myPal.aunPalette[0] = index_0;
15. myPal.aunPalette[1] = index_1;
16. myPal.u16PalStart = 0
17. myPal.u16PalEnd = 1;
18. MI_GFX_SetPalette(DEFAULT_GFX_DEV_ID, E_MI_GFX_FMT_I8, &myPal);
```

2.9. MI_GFX_CreateDev

➤ 功能

初始化 gfx 设备。

➤ 语法

MI_S32 MI_GFX_CreateDev([MI_GFX_DEV](#) GfxDevId, [MI_GFX_DevAttr t](#) *pstDevAttr);

➤ 形参

参数名称	描述	输入/输出
GfxDevId	GFX设备号	输入
pstDevAttr	设备初始化参数。	输入

➤ 返回值

返回值 {
· MI_SUCCESS 成功。
· 失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_gfx.h mi_gfx_datatype.h
库文件: libmi_gfx.a/libmi_gfx.so

※ 注意

无。

➤ 举例

无。

2.10. MI_GFX_DestroyDev

➤ 功能

反初始gfx设备。

➤ 语法

MI_S32 MI_GFX_DestroyDev([MI_GFX_DEV](#) GfxDevId);

➤ 形参

参数名称	描述	输入/输出
GfxDevId	GFX设备号	输入

➤ 返回值

返回值 {
· MI_SUCCESS 成功。
· 失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_gfx.h mi_gfx_datatype.h
库文件: libmi_gfx.a/libmi_gfx.so

※ 注意

- 此函数必须在初始化设备后调用，否则返回失败。
- 如果本接口在 **app** 退出前没有调用，内部会自动反初始化设备。

➤ 举例

无。

2.11. MI_GFX_GetARGB1555To8888AlphaValue

➤ 功能

获取 alpha 转换值。

当源位图和目标位图做 bitblit 操作时，不管源位图和目标位图的像素格式是什么，硬件都会各生成一个像素格式为 ARGB8888 的中间位图，再将中间位图的运算结果输出到目标位图。当源位图或目标图片像素格式为 ARGB1555 时，若ARGB1555的 alpha 位为0，则转换后ARGB8888的 alpha 为 0；若ARGB1555的 alpha 位为1，则转换后ARGB8888的 alpha 为 alpha 转换值。

➤ 语法

```
MI_S32 MI_GFX_GetARGB1555To8888AlphaValue(MI_GFX_DEV GfxDevId, MI_U8 *pu8AlphaValue);
```

➤ 形参

参数名称	描述	输入/输出
GfxDevId	GFX设备号	输入
pu8AlphaValue	alpha 转换值	输出

➤ 返回值

返回值 {
· MI_SUCCESS 成功。
· 获取alpha转换值失败，详情参照[错误码](#)。

➤ 依赖

头文件：mi_gfx.h mi_gfx_datatype.h
库文件：libmi_gfx.a/libmi_gfx.so

※ 注意

无。

➤ 举例

无。

2.12. MI_GFX_SetARGB1555To8888AlphaValue

➤ 功能

设置 alpha 转换值。

当源位图和目标位图做 bitblit 操作时，不管源位图和目标位图的像素格式是什么，硬件都会各生成一个像素格式为 ARGB8888 的中间位图，再将中间位图的运算结果输出到目标位图。当源位图或目标图片像素格式为 ARGB1555 时，若ARGB1555的 alpha 位为0，则转换后ARGB8888的 alpha 为 0；若ARGB1555的 alpha 位为1，则转换后ARGB8888的 alpha 为 alpha 转换值。

➤ 语法

```
MI_S32 MI_GFX_SetARGB1555To8888AlphaValue(MI_GFX_DEV GfxDevId, MI_U8 u8AlphaValue);
```

➤ 形参

参数名称	描述	输入/输出
GfxDevId	GFX设备号	输入
u8AlphaValue	alpha 转换值	输入

➤ 返回值

返回值 {
· MI_SUCCESS 成功。
· 获取alpha转换值失败, 详情参照[错误码](#)。

➤ 依赖

头文件: `mi_gfx.h` `mi_gfx_datatype.h`
库文件: `libmi_gfx.a/libmi_gfx.so`

※ 注意

无。

➤ 举例

无。

2.13. MI_GFX_GetInitialScalingCoeff

➤ 功能

获取缩放系数。当 `bitblit` 操作需要缩放时, 双线性插值的图像几何中心偏移=缩放系数/1000。

➤ 语法

`MI_S32 MI_GFX_GetInitialScalingCoeff(MI\_GFX\_DEV GfxDevId, MI_U16 *pu16ScalingCoefficient);`

➤ 形参

参数名称	描述	输入/输出
<code>GfxDevId</code>	GFX设备号	输入
<code>pu16ScalingCoefficient</code>	缩放系数	输出

➤ 返回值

返回值 {
· MI_SUCCESS 成功。
· 获取缩放系数失败, 详情参照[错误码](#)。

➤ 依赖

头文件: `mi_gfx.h` `mi_gfx_datatype.h`
库文件: `libmi_gfx.a/libmi_gfx.so`

※ 注意

无。

➤ 举例

无。

2.14. MI_GFX_SetInitialScalingCoeff

➤ 功能

设置缩放系数。当 bitblit 操作需要缩放时，双线性插值的图像几何中心偏移=缩放系数/1000。默认值为500。

➤ 语法

MI_S32 MI_GFX_SetInitialScalingCoeff([MI_GFX_DEV](#) GfxDevId, MI_U16 u16ScalingCoefficient);

➤ 形参

参数名称	描述	输入/输出
GfxDevId	GFX设备号	输入
u16ScalingCoefficient	缩放系数	输入

➤ 返回值

返回值 {
· MI_SUCCESS 成功。
· 设置缩放系数失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_gfx.h mi_gfx_datatype.h
库文件: libmi_gfx.a/libmi_gfx.so

※ 注意

无。

➤ 举例

无。

2.15. MI_GFX_DrawLine

➤ 功能

向任务中加入画线操作。

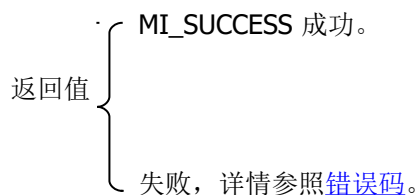
➤ 语法

MI_S32 MI_GFX_DrawLine([MI_GFX_Surface_t](#) *pstDst, [MI_GFX_Line_t](#) *pstLine, MI_U16 *pu16Fence);

➤ 形参

参数名称	描述	输入/输出
pstDst	目标位图。	输入
pstLine	直线属性	输入
pu16Fence	返回Fence的指针	输出

➤ 返回值



➤ 依赖

头文件: `mi_gfx.h`
库文件: `libmi_gfx.a/libmi_gfx.so`

※ 注意

- 在调用此接口前应保证调用 [MI_GFX_Open](#) 打开 GFX 设备。
- 画线区域不能超出目标位图的大小。
- 直线的终点不会被画出。
- 如果是目标位图的像素格式为RGB或ARGB, `u32ColorFrom/u32ColorFrom`从高位到低位, 依次是A8: R8: G8: B8, 如果目标位图的像素格式为YUYV422, 则A8则无效, R对应Y, G对应U, B对应V。

➤ 举例

```

1. #define DEFAULT_GFX_DEV_ID (0)
2. #define DEFAULT_SOC_ID (0)
3.
4. FILE *fp = NULL;
5. MI_GFX_Surface_t stDst;
6. MI_GFX_Line_t stLine;
7. MI_U16 u16TargetFence;
8. MI_PHY phyAddr;
9. void *VirAddr;
10.
11. ExecFunc(MI_SYS_Init(DEFAULT_SOC_ID), MI_SUCCESS);
12. fp = fopen(SRC_FILE_NAME, "wb");
13. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0", SRC_WIDTH*SRC_HEIGHT*2, &phyAddr), MI_SUCCESS);
14.
15. //MI_U32 phySrcAddr, phyDstAddr;
16. ExecFunc(MI_GFX_Open(DEFAULT_SOC_ID), MI_SUCCESS);
17. ExecFunc(MI_SYS_Mmap(phyAddr, SRC_WIDTH*SRC_HEIGHT*2, &pVirAddr, FALSE), MI_SUCCESS);
18. memset(pVirAddr, 0x22, SRC_WIDTH*SRC_HEIGHT*2);
19.
20. //draw line
21. memset(&stDst, 0x0, sizeof(stDst));
22. stDst.eColorFmt = E_MI_GFX_FMT_ARGB1555;
23. stDst.u32Width = SRC_WIDTH;
24. stDst.u32Height = SRC_HEIGHT;
25. stDst.u32Stride = SRC_WIDTH * 2;
26. stDst.phyAddr = phyAddr;
27.
28. memset(&stLine, 0x0, sizeof(stLine));
29. stLine.stPointFrom.s16x = 0;
30. stLine.stPointFrom.s16y = 0;
31. stLine.stPointTo.s16x = 100;
32. stLine.stPointTo.s16y = 100;
33. stLine.u16Width = 1;
34. stLine.bColorGradient = TRUE;
35. stLine.bColorFrom = 0xffff0000;
36. stLine.bColorTo = 0xffff00ff;
  
```

```
37.  
38. ExecFunc(MI_GFX_DrawLine(DEFAULT_SOC_ID, &stDst, &stLine, &u16TargetFence), MI_SUCCESS);  
39. ExecFunc(MI_GFX_WaitAllDone(DEFAULT_SOC_ID, FALSE, u16TargetFence), MI_SUCCESS);  
40. if (NULL != fp)  
41. {  
42.     fwrite(pVirAddr, 1, SRC_WIDTH*SRC_HEIGHT*2, fp);  
43.     fclose(fp);  
44.     fp = NULL;  
45. }  
46. ExecFunc(MI_SYS_Munmap(pVirAddr, SRC_WIDTH*SRC_HEIGHT*2), MI_SUCCESS);  
47. ExecFunc(MI_SYS_MMA_Free(phyAddr), MI_SUCCESS);  
48.  
49. ExecFunc(MI_GFX_Close(DEFAULT_SOC_ID), MI_SUCCESS);  
50. ExecFunc(MI_SYS_Exit(DEFAULT_SOC_ID), MI_SUCCESS);
```

3. GFX 数据类型

GFX 相关数据类型如下表：

数据类型	说明
MI_GFX_ColorFmt_e	定义像素颜色格式
MI_GFX_Surface_t	定义位图属性结构体
MI_GFX_Rect_t	定义操作区域属性结构体
MI_GFX_ColorKeyOp_e	定义Colorkey 操作模式
MI_GFX_ColorKeyValue_t	定义Colorkey 颜色属性结构体
MI_GFX_ColorKeyInfo_t	定义Colorkey 属性结构体
MI_GFX_Rotate_e	定义图像旋转角度
MI_GFX_Mirror_e	定义图像镜像模式
MI_GFX_DfbBldOp_e	定义Alpha Blending 的模式
MI_Gfx_DfbBlendFlags_e	定义Alpha Blending 的标志
MI_GFX_Opt_t	定义Bitblit 操作的可选属性结构体
MI_GFX_PaletteEntry_t	定义调色板颜色结构体
MI_GFX_Palette_t	定义调色板结构体
MI_GFX_DevAttr_t	定义GFX设备初始化参数
MI_GFX_Point_t	定义点属性结构体
MI_GFX_Line_t	定义直线属性结构体
MI_GFX_DEV	GFX的设备号

3.1. MI_GFX_ColorFmt_e

➤ 说明

定义像素颜色格式。

➤ 定义

```
typedef enum
{
    E_MI_GFX_FMT_I1 = 0, /* ColorFormat */
    E_MI_GFX_FMT_I2,
    E_MI_GFX_FMT_I4,
    E_MI_GFX_FMT_I8,
    E_MI_GFX_FMT_FABAFGBG2266,
    E_MI_GFX_FMT_1ABFGBG12355,
    E_MI_GFX_FMT_RGB565,
    E_MI_GFX_FMT_ARGB1555,
    E_MI_GFX_FMT_ARGB4444,
    E_MI_GFX_FMT_ARGB1555_DST,
    E_MI_GFX_FMT_ARGB8888,
    E_MI_GFX_FMT_RGBA5551,
    E_MI_GFX_FMT_RGBA4444,
    E_MI_GFX_FMT_ABGR8888,
    E_MI_GFX_FMT_BGRA5551,
    E_MI_GFX_FMT_ABGR1555,
    E_MI_GFX_FMT_ABGR4444,
    E_MI_GFX_FMT_BGRA4444,
    E_MI_GFX_FMT_BGR565,
    E_MI_GFX_FMT_RGBA8888,
    E_MI_GFX_FMT_BGRA8888,
    E_MI_GFX_FMT_YUV422_YUYV,
    E_MI_GFX_FMT_YUV422_YVYU,
    E_MI_GFX_FMT_YUV422_UYVY,
    E_MI_GFX_FMT_YUV422_VYUY,
    E_MI_GFX_FMT_MAX
} MI_GFX_ColorFmt_e;
```

➤ 注意

对应 surface 支持的 color format，在 bitblit 操作时，对应 SrcSurface、DstSurface 的颜色格式。I1/I2/I4/I8 不能作为 DstSurface 的颜色格式。

➤ 相关数据类型及接口

[MI_GFX_ColorKeyInfo_t](#) , [MI_GFX_Surface_t](#) , [MI_GFX_SetPalette](#)

3.2. MI_GFX_Surface_t

➤ 说明

定义位图属性结构体。

➤ 定义

```
typedef struct MI_GFX_Surface_s
{
```

```
MI_PHY phyAddr;
MI_GFX_ColorFmt_e eColorFmt;
MI_U32 u32Width;
MI_U32 u32Height;
MI_U32 u32Stride;
} MI_GFX_Surface_t;
```

➤ 成员

成员	描述
u32PhyAddr	位图对应内存的物理地址。
eColorFmt	位图像素颜色格式。
u32Height	位图高度。
u32Width	位图宽度。
u32Stride	位图跨度。

➤ 注意事项

- 当位图的像素格式大小大于或等于 1Byte 时，位图对应内存的物理地址和 Stride 必须按照像素格式大小对齐；当位图的像素格式大小不足 1Byte 时，位图对应内存的物理地址和 Stride 需要按照 1Byte 对齐。
- YUYV422 格式的位图的水平起始位置和宽度必须为偶数。

➤ 相关数据类型及接口

[MI_GFX_QuickFill](#) , [MI_GFX_DrawLine](#) , [MI_GFX_BitBlit](#)

3.3. MI_GFX_Rect_t

➤ 说明

定义操作区域结构体。

➤ 定义

```
typedef struct MI_GFX_Rect_s
{
    MI_S32 s32Xpos;
    MI_S32 s32Ypos;
    MI_U32 u32Width;
    MI_U32 u32Height;
} MI_GFX_Rect_t;
```

➤ 成员

成员	描述
s32Xpos	操作区域的起始横坐标，以像素数为单位。有效范围[0, 位图宽度)
s32Ypos	操作区域的起始纵坐标，以像素数为单位。有效范围[0, 位图高度)
u32Width	操作区域的宽度，以像素数为单位。
u32Height	操作区域的高度，以像素数为单位。

※ 注意事项

操作范围必须在位图范围内。

- 相关数据类型及接口

[MI_GFX_Opt_t](#) , [MI_GFX_QuickFill](#) , [MI_GFX_BitBlit](#)

3.4. MI_GFX_ColorKeyOp_e

- 说明

定义 colorkey 操作模式。

- 定义

```
typedef enum
{
    E_MI_GFX_RGB_OP_EQUAL = 0,
    E_MI_GFX_RGB_OP_NOT_EQUAL,
    E_MI_GFX_ALPHA_OP_EQUAL,
    E_MI_GFX_ALPHA_OP_NOT_EQUAL,
    E_MI_GFX_ARGB_OP_EQUAL,
    E_MI_GFX_ARGB_OP_NOT_EQUAL,
    E_MI_GFX_CKEY_OP_BUTT,
} MI_GFX_ColorKey_e;
```

- 成员

成员	描述
E_MI_GFX_RGB_OP_EQUAL	RGB 相等做 colorkey 操作
E_MI_GFX_RGB_OP_NOT_EQUAL	RGB 不相等做 colorkey 操作
E_MI_GFX_ALPHA_OP_EQUAL	ALPHA 相等做 colorkey 操作
E_MI_GFX_ALPHA_OP_NOT_EQUAL	ALPHA 不相等做 colorkey 操作
E_MI_GFX_ARGB_OP_EQUAL	ARGB 相等做 colorkey 操作，详见注意事项
E_MI_GFX_ARGB_OP_NOT_EQUAL	ARGB 不相等做 colorkey 操作，详见注意事项
E_MI_GFX_CKEY_OP_BUTT	无效的 colorkey 操作

- ※ 注意事项

- 对位图进行 colorkey 时，可选择前景、背景，也可以选择key值相等或者不等。
- E_MI_GFX_ARGB_OP_EQUAL的选中条件如下：
Ikayaki系列：
srcSurface: (Apixel == Acolorkey) || (RGBpixel == RGBcolorkey)
dstSurface: (ARGBpixel == ARGBcolorkey)
Muffin、Mochi系列: (ARGBpixel == ARGBcolorkey)
- E_MI_GFX_ARGB_OP_NOT_EQUAL的选中条件如下：
Ikayaki系列：
srcSurface: (ARGBpixel != ARGBcolorkey)
dstSurface: (Apixel != Acolorkey) || (RGBpixel != RGBcolorkey)
Muffin、Mochi系列：
(ARGBpixel != ARGBcolorkey)

- 相关数据类型及接口

[MI_GFX_ColorKeyInfo_t](#)

3.5. MI_GFX_ColorKeyValue_t

➤ 说明

定义 colorkey 的颜色结构体。

➤ 定义

```
typedef struct MI_GFX_ColorKey_s
{
    MI_U32 u32ColorStart;
    MI_U32 u32ColorEnd;
} MI_GFX_ColorKeyValue_t;
```

➤ 成员

成员	描述
u32ColorStart	ColorKey颜色范围起始值
u32ColorEnd	ColorKey颜色范围结束值

※ 注意事项

当仅针对一个 color 值做 colorkey 操作时，将 u32ColorStart = u32ColorEnd = colorVal。

➤ 相关数据类型及接口

[MI_GFX_ColorKeyInfo_t](#)

3.6. MI_GFX_ColorKeyInfo_t

➤ 说明

定义 colorkey 属性结构体。

➤ 定义

```
typedef struct MI_GFX_ColorKeyInfo_s
{
    MI_BOOL bEnColorKey;
    MI\_GFX\_ColorKeyOp\_e eCKeyOp;
    MI\_GFX\_ColorFmt\_e eCKeyFmt;
    MI\_GFX\_ColorKeyValue\_t stCKeyVal;
} MI_GFX_ColorKeyInfo_t;
```

➤ 成员

成员	描述
bEnColorKey	是否使能ColorKey操作
eCKeyOp	ColorKey操作模式
stCKeyVal	ColorKey颜色范围
eCKeyFmt	ColorKey的颜色格式

※ 注意事项

无。

➤ 相关数据类型及接口

[MI_GFX_Opt_t](#)

3.7. MI_GFX_Rotate_e

➤ 说明

定义图像旋转角度。

➤ 定义

```
typedef enum
{
    E_MI_GFX_ROTATE_0 = 0,
    E_MI_GFX_ROTATE_90,
    E_MI_GFX_ROTATE_180,
    E_MI_GFX_ROTATE_270
} MI_GFX_Rotate_e;
```

➤ 成员

无。

※ 注意事项

无。

➤ 相关数据类型及接口

[MI_GFX_Opt_t](#)

3.8. MI_GFX_Mirror_e

➤ 说明

定义图像镜像模式。

➤ 定义

```
typedef enum
{
    E_MI_GFX_MIRROR_NONE = 0,
    E_MI_GFX_MIRROR_HORIZONTAL,
    E_MI_GFX_MIRROR_VERTICAL,
    E_MI_GFX_MIRROR_BOTH,
    E_MI_GFX_MIRROR_MAX
} MI_GFX_Mirror_e;
```

➤ 成员

成员	描述
E_MI_GFX_MIRROR_NONE	输出图像不进行镜像操作
E_MI_GFX_MIRROR_HORIZONTAL	输出图像水平镜像
E_MI_GFX_MIRROR_VERTICAL	输出图像垂直镜像
E_MI_GFX_MIRROR_BOTH	输出图像水平+垂直镜像
E_MI_GFX_MIRROR_MAX	无效的镜像类型

※ 注意事项

无。

➤ 相关数据类型及接口

[MI_GFX_Opt_t](#)

3.9. MI_GFX_DfbBldOp_e

➤ 说明

定义 Alpha Blending 的模式。

➤ 定义

```
typedef enum
{
    E_MI_GFX_DFB_BLD_ZERO = 0,
    E_MI_GFX_DFB_BLD_ONE,
    E_MI_GFX_DFB_BLD_SRCCOLOR,
    E_MI_GFX_DFB_BLD_INVSRCOLOR,
    E_MI_GFX_DFB_BLD_SRCALPHA,
    E_MI_GFX_DFB_BLD_INVSRCALPHA,
    E_MI_GFX_DFB_BLD_DESTALPHA,
    E_MI_GFX_DFB_BLD_INVDESTALPHA,
    E_MI_GFX_DFB_BLD_DESTCOLOR,
    E_MI_GFX_DFB_BLD_INVDESTCOLOR,
    E_MI_GFX_DFB_BLD_SRCALPHASAT,
    E_MI_GFX_DFB_BLD_NONE,
    E_MI_GFX_DFB_BLD_MAX,
} MI_GFX_DfbBldOp_e;
```

➤ 成员

成员	描述
E_MI_GFX_DFB_BLD_OP_ZERO	argb *= 0.0 同directfb DSBF_ZERO
E_MI_GFX_DFB_BLD_OP_ONE	argb *= 1.0 同directfb DSBF_ONE
E_MI_GFX_DFB_BLD_OP_SRCCOLOR	argb *= Sargb 同directfb DSBF_SRCCOLOR
E_MI_GFX_DFB_BLD_OP_INVSRCOLOR	argb *= 1.0 - Sargb 同directfb DSBF_INVSRCOLOR
E_MI_GFX_DFB_BLD_OP_SRCALPHA	argb *= Saaaa 同directfb DSBF_SRCALPHA
E_MI_GFX_DFB_BLD_OP_INVSRCALPHA	argb *= 1.0 - Saaaa 同directfb DSBF_INVSRCALPHA
E_MI_GFX_DFB_BLD_OP_DESTCOLOR	argb *= Dargb 同directfb DSBF_DESTCOLOR
E_MI_GFX_DFB_BLD_OP_INVDESTCOLOR	argb *= 1.0 - Dargb 同directfb DSBF_INVDESTCOLOR
E_MI_GFX_DFB_BLD_OP_DESTALPHA	argb *= Daaaa

成员	描述
	同directfb DSBF_DESTALPHA
E_MI_GFX_DFB_BLD_OP_INVDESTALPHA	argb *= 1.0 – Daaaa 同directfb DSBF_INVDESTALPHA
E_MI_GFX_DFB_BLD_OP_SRCALPHASAT	rgb *= min(Sa, 1-Da) 同directfb DSBF_SRCALPHASAT
E_MI_GFX_DFB_BLD_NONE	无效的 alpha 混合模式
E_MI_GFX_BLD_MAX	argb *= 0.0 同directfb DSBF_ZERO

※ 注意事项

在前景位图和背景位图作叠加运算时，可以分别设置 Src 通道和 Dst 通道的叠加模式。现在支持 11 种叠加模式。用 MI_GFX_DfbBldOp_e 设置模式。

➤ 相关数据类型及接口

[MI_GFX_Opt_t](#)

3.10. MI_Gfx_DfbBlendFlags_e

➤ 说明

定义 Alpha Blending 的标志。

➤ 定义

```
typedef enum
{
    E_MI_GFX_DFB_BLEND_NOFX = 0x00000000,
    E_MI_GFX_DFB_BLEND_COLORALPHA = 0x00000001,
    E_MI_GFX_DFB_BLEND_ALPHACHANNEL = 0x00000002,
    E_MI_GFX_DFB_BLEND_COLORIZE = 0x00000004,
    E_MI_GFX_DFB_BLEND_SRC_PREMULTIPLY = 0x00000008,
    E_MI_GFX_DFB_BLEND_SRC_PREMULTCOLOR = 0x00000010,
    E_MI_GFX_DFB_BLEND_DST_PREMULTIPLY = 0x00000020,
    E_MI_GFX_DFB_BLEND_XOR = 0x00000040,
    E_MI_GFX_DFB_BLEND_DEMULTIPLY = 0x00000080,
    E_MI_GFX_DFB_BLEND_SRC_COLORKEY = 0x00000100,
    E_MI_GFX_DFB_BLEND_DST_COLORKEY = 0x00000200,
    E_MI_GFX_DFB_BLEND_MAX = 0x3FF
} MI_Gfx_DfbBlendFlags_e;
```

➤ 成员

成员	描述
E_MI_GFX_DFB_BLEND_NOFX	没有任何BLD操作
E_MI_GFX_DFB_BLEND_COLORALPHA	src alpha会根据const color value运算 src.a = const color.a;

	同directfb DSBLIT_BLEND_COLORALPHA
E_MI_GFX_DFB_BLEND_ALPHACHANNEL	src alpha会根据const color value运算 $\text{src.a} = \text{src.a} * \text{const color.a};$ 同directfb DSBLIT_BLEND_ALPHACHANNEL。
E_MI_GFX_DFB_BLEND_COLORIZE	将const color作为全局颜色，调整源颜色 同directfb DSBLIT_COLORIZE。
E_MI_GFX_DFB_BLEND_SRC_PREMULTIPLY	使用源alpha预乘源颜色 $\text{src.r} = \text{src.r} * \text{src.a};$ 等 同directfb DSBLIT_SRC_PREMULTIPLY。
E_MI_GFX_DFB_BLEND_SRC_PREMULTICOLOR	使用const alpha预乘源颜色 $\text{src.r} = \text{src.r} * \text{const.a};$ 等 同directfb DSBLIT_SRC_PREMULTICOLOR。
E_MI_GFX_DFB_BLEND_DST_PREMULTIPLY	使用目的alpha预乘目的颜色 $\text{dst.r} = \text{dst.r} * \text{dst.a};$ 等 同directfb DSBLIT_DST_PREMULTIPLY。
E_MI_GFX_DFB_BLEND_XOR	调节和预乘alpha $\text{src.a} \wedge= \text{dst.a};$ 同directfb DSBLIT_XOR。
E_MI_GFX_DFB_BLEND_DEMULTIPLY	预乘到非预乘的转换，最好不要使用这样的操作 $\text{x.r} = ((\text{int})\text{x.r} < 8) / ((\text{int})\text{x.a} + 1);$ 等 同directfb DSBLIT_DEMULTIPLY。
E_MI_GFX_DFB_BLEND_SRC_COLORKEY	如果src的颜色等于const color，则会被key掉
E_MI_GFX_DFB_BLEND_DST_COLORKEY	如果dst的颜色等于const color，则会被key掉
E_MI_GFX_DFB_BLEND_MAX	上面所有的action都会被使能

➤ 相关数据类型及接口

[MI_GFX_Opt_t](#)

3.11. MI_GFX_BitBltMode_e

➤ 说明

定义 BitBlt 的插值模式。

➤ 定义

```
typedef enum
{
    E_MI_GFX_BITBLT_MODE_BILINEAR,
    E_MI_GFX_BITBLT_MODE_NEAREST,
} MI_GFX_BitBltMode_e;
```

➤ 成员

成员	描述
E_MI_GFX_BITBLT_MODE_BILINEAR	双线性插值模式
E_MI_GFX_BITBLT_MODE_NEAREST	邻近插值模式

- 相关数据类型及接口

[MI_GFX_Opt_t](#)

3.12. MI_GFX_Opt_t

- 说明

定义 BitBlit 操作的可选属性结构体。

- 定义

```
typedef struct MI_GFX_Opt_s
{
    MI\_GFX\_Rect\_t stClipRect;
    MI\_GFX\_ColorKeyInfo\_t stSrcColorKeyInfo;
    MI\_GFX\_ColorKeyInfo\_t stDstColorKeyInfo;
    MI\_GFX\_DfbBldOp\_e eSrcDfbBldOp;
    MI\_GFX\_DfbBldOp\_e eDstDfbBldOp;
    MI\_GFX\_Mirror\_e eMirror;
    MI\_GFX\_Rotate\_e eRotate;
    MI\_Gfx\_DfbBlendFlags\_e eDFBBlendFlag;
    MI\_GFX\_BitBltMode\_e eMode;
    MI_U32 u32GlobalSrcConstColor;
    MI_U32 u32GlobalDstConstColor;
} MI_GFX_Opt_t;
```

- 成员

成员	描述
stSrcColorKeyInfo	Src surface colorkey 操作
stDstColorKeyInfo	Dst surface colorkey操作
stClipRect	clip 区域定义
eMirror	图像镜像类型
eSrcDfbBldOp	Src Surface BLEND操作类型
eDstDfbBldOp	Dst Surface BLEND操作类型
eRotate	图像旋转角度，0、90、180、270度旋转
eDFBBlendFlag	Blending的相关操作，如XOR、AlphaBlending、Colorize等操作
eMode	Bitblt使用的插值模式
u32GlobalSrcConstColor	针对eDFBBlendFlag 操作对应的const参数，eDFBBlendFlag 操作有对应的mutli等等
u32GlobalDstConstColor	针对eDFBBlendFlag 操作对应的const参数，eDFBBlendFlag 操作有对应的mutli等等

- ※ 注意事项

目前仅有 u32GlobalSrcConstColor 有效。

- 相关数据类型及接口

[MI_GFX_BitBlit](#)

3.13. MI_GFX_PaletteEntry_t

➤ 说明

定义调色板颜色结构体。

➤ 定义

```
typedef union
{
    /// ARGB8888 byte order
    struct
    {
        MI_U8 u8A;
        MI_U8 u8R;
        MI_U8 u8G;
        MI_U8 u8B;
    } RGB;
    // u8Data[0] = u8A
    // u8Data[1] = u8R
    // u8Data[2] = u8G
    // u8Data[3] = u8B
    MI_U8 u8Data[4];
} MI_GFX_PaletteEntry_t;
```

➤ 成员

成员	描述
RGB	索引颜色对应的RGB颜色的分量结构体表示
u8Data	索引颜色对应的RGB颜色的字节序表示

※ 注意事项

注意定义中注释对颜色定义的描述，以便正确填写调色板数据。

➤ 相关数据类型及结果

[MI_GFX_Palette_t](#)

3.14. MI_GFX_Palette_t

➤ 说明

定义调色板结构体。

➤ 定义

```
typedef struct MI_GFX_Palette_s
{
    MI\_GFX\_PaletteEntry\_t aunPalette[256];
    MI_U16 u16PalStart;
```

```
MI_U16 u16PalEnd;
}MI_GFX_Palette_t;
```

➤ 成员

成员	描述
aunPalette	调色板索引对应的RGB颜色数组
u16PalStart	需要配置RGB颜色的索引集起始索引
u16PalEnd	需要配置RGB颜色的索引集结束索引

※ 注意事项

无。

➤ 相关数据类型及结果

[MI_GFX_PaletteEntry_t](#)

3.15. MI_GFX_DevAttr_t

➤ 说明

GFX 设备初始化参数。

➤ 定义

```
typedef struct MI_GFX_DevAttr_s
{
    MI_U32 u32DevId;
    MI_U8 *u8Data;
} MI_GFX_DevAttr_t;
```

➤ 成员

成员名称	描述
u32DevId	设备 ID
u8Data	数据指针 buffer

➤ 注意事项

无。

➤ 相关数据类型及接口

[MI_GFX_CreateDev](#)

3.16. MI_GFX_Point_t

➤ 说明

定义点属性结构体。

➤ 定义

```
typedef struct MI_GFX_Point_s
{
    MI_S16 s16x;
    MI_S16 s16y;
} MI_GFX_Point_t;
```

➤ 成员

成员名称	描述
s16x	X 坐标
S16y	Y 坐标

➤ 注意事项

无。

➤ 相关数据类型及接口

[MI_GFX_Line_t](#)

3.17. MI_GFX_Line_t

➤ 说明

定义直线属性结构体。

➤ 定义

```
typedef struct MI_GFX_Line_s
{
    MI\_GFX\_Point\_t stPointFrom;
    MI\_GFX\_Point\_t srPointTo;
    MI_U16 u16Width;
    MI_BOOL bColorGradient;
    MI_U32 u32ColorFrom;
    MI_U32 u32ColorTo;
} MI_GFX_Line_t;
```

➤ 成员

成员名称	描述
stPointFrom	直线的起点
srPointTo	直线的终点
u16Width	直线宽度
bColorGradient	是否颜色渐变
u32ColorFrom	渐变起始颜色

成员名称	描述
u32ColorTo	渐变结束颜色

➤ 注意事项

无。

➤ 相关数据类型及接口

[MI_GFX_DrawLine](#)

3.18. MI_GFX_DEV

➤ 说明

GFX 设备号。

➤ 定义

```
typedef MI_S32 MI_GFX_DEV;
```

➤ 成员

无

➤ 注意事项

除 Muffin 支持 2 个 device，其他系列芯片均只支持 1 个 device。

➤ 相关数据类型及接口

所有的接口。

4. GFX 错误码

GFX API 错误码如下所示

错误代码	宏定义	描述
0xA00D2003	MI_ERR_GFX_INVALID_PARAM	非法参数
0xA00D2002	MI_ERR_GFX_DEV_BUSY	GFX设备忙
0xA00D2220	MI_ERR_GFX_NOT_INIT	设备未初始化
0xA00D2221	MI_ERR_GFX_DRV_NOT_SUPPORT	不支持的操作类型
0xA00D2222	MI_ERR_GFX_DRV_FAIL_FORMAT	不支持的格式类型
0xA00D2223	MI_ERR_GFX_NON_ALIGN_ADDRESS	地址未对齐
0xA00D2224	MI_ERR_GFX_NON_ALIGN_PITCH	Pitch对齐错误
0xA00D2225	MI_ERR_GFX_DRV_FAIL_OVERLAP	Option内存有叠加区域
0xA00D2226	MI_ERR_GFX_DRV_FAIL_STRETCH	缩放失败
0xA00D2228	MI_ERR_GFX_DRV_FAIL_LOCKED	设备被其他使用者锁住
0xA00D2229	MI_ERR_GFX_DRV_FAIL_BLTADDR	Bitblit起始地址错误

5. PROCFS 介绍

5.1. cat

➤ 调试信息

cat /proc/mi_modules/mi_gfx/mi_gfx0

```
-----Gfx Dev Attr-----
module init count:  2
Device id:  0
ARGB1555 alpha threshold:  1
ARGB1555 To ARGB8888 alpha value:  255
Scaling coefficient:  500 / 1000

Device id:  1
ARGB1555 alpha threshold:  1
ARGB1555 To ARGB8888 alpha value:  255
Scaling coefficient:  500 / 1000
-----
```

➤ 调试信息分析

记录当前 GFX 的使用状况以及 device 属性可以动态地获取到这些信息，方便调试和测试。

➤ 参数说明

参数		描述
device info	module init count	设备被 Open 的次数，由于 GFX 会有多个模块进行调用，并且不能确定是哪一个 module 会先调用 GFX 的初始化，所以设备允许多次 Open，会记录 Open 次数来在 Close 的时候做对应的计数判断是否需要真的 Close Device。
	Device id	GFX 的设备 ID
	ARGB1555 alpha threshold	ARGB8888 转 ARGB1555 时，GFX 内部的 alpha 判定阈值，如大于 N 则 alpha 为 1，反之 alpha 为 0。
	ARGB1555 To ARGB8888 alpha value	ARGB1555 转 ARGB8888 时，GFX 将 ARGB1555 alpha bit 为 1 的像素的 alpha 转为 ARGB1555 To ARGB8888 alpha value
	Scaling coefficient	图像几何中心偏移